DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3106-02 - APPLICATION DEVELOPMENT

REPORT

RESTAURANT ORDER MANAGEMENT SYSTEM

NAME	MATRIC NO
JEANETTE HAUW CHANDRA	X25EC3020

Presented for: Dr. Seah Choon Sen

18 July 2026

Introduction

This report provides a comprehensive overview of the design, development, and implementation of the "Restaurant Order Management System." In the contemporary food and beverage industry, the shift toward digitized operational workflows has become essential for maintaining competitiveness and ensuring high standards of service. Many traditional restaurant settings continue to rely on manual or paper-based order tracking, which often leads to communication bottlenecks, delayed preparation times, and increased risks of administrative errors during the payment phase.

To address these challenges, this project introduces a centralized, cloud-integrated digital platform designed to facilitate the entire order lifecycle. The system is engineered to manage real-time updates across multiple stages of an order's progression, specifically tracking orders from their initial "Pending" status, through the culinary preparation phase, to the serving stage, and finally, to the conclusion of the transaction via payment processing. By leveraging modern mobile-first development frameworks and robust backend-as-a-service architecture, this system ensures that restaurant staff can monitor table-specific demands with high accuracy and minimal latency. This report further details the technological stack utilized, the iterative development process, and the structural interface design implemented to optimize the efficiency of front-of-house and back-of-house operations.

Software, Language, Tools, API, and Templates Used

- **Programming Language:** The project is written in Dart, an object-oriented language selected for its high performance and seamless integration with the Flutter framework.
- **Framework:** Flutter was utilized as the primary development framework, enabling a single codebase to produce a responsive application capable of functioning across web and mobile platforms with consistent behavior.
- **Backend & Database:** Supabase was deployed as the Backend-as-a-Service (BaaS) provider, offering a scalable PostgreSQL database, real-time subscription capabilities, and secure API endpoints that eliminate the need for traditional server-side infrastructure.
- **Development Environment:** Visual Studio Code served as the primary Integrated Development Environment (IDE), chosen for its extensive plugin ecosystem and robust debugging tools that streamlined the coding process.
- **API/Client SDK:** The project utilizes the Supabase Flutter Client SDK, which acts as the intermediary bridge for data retrieval and modification between the application and the cloud database.
- **UI/UX Design:** For the system interface, the application implemented a simple monochrome UI design. This aesthetic choice focuses on clarity and readability, utilizing a clean, high-contrast palette to ensure that staff can focus entirely on operational tasks without distractions from overly complex color schemes or decorative elements.

Development Steps

1. **Project Initialization:** The development process began with configuring the Flutter environment. This involved establishing the project structure, defining the necessary dependencies, and initializing the Supabase client to create a secure, authenticated channel between the app and the cloud database.
2. **Database Schema Design:** A robust relational database schema was designed within Supabase. I defined the orders table to track global state, such as table numbers and payment

status, and a linked order_items table to handle specific menu selections, ensuring data integrity through relational keys.

3. **Model Implementation:** To ensure reliable data handling, I engineered an Order data model. This component acts as the bridge between raw database JSON and the application's objects, incorporating strict null-safety mechanisms that prevent runtime crashes when dealing with missing or incomplete data packets.
4. **Service Layer Development:** I developed a dedicated OrderService class to encapsulate all database interaction logic. This layer abstracts complex API calls into simple, reusable methods, utilizing asynchronous programming to keep the user interface responsive during heavy database fetch and update operations.
5. **Interface Construction:** The development of the Order List and Detail screens involved mapping the data model to graphical widgets. I implemented business logic that dictates how the UI responds to state changes, such as preventing users from deleting orders that have already progressed past the "Pending" phase.
6. **Optimization & Refinement:** The final phase involved iterative testing to ensure the application performed correctly within web browsers. This included fixing type-safety errors unique to web rendering and polishing the layout spacing to ensure that buttons and text remain accessible across various screen sizes.

System Interface

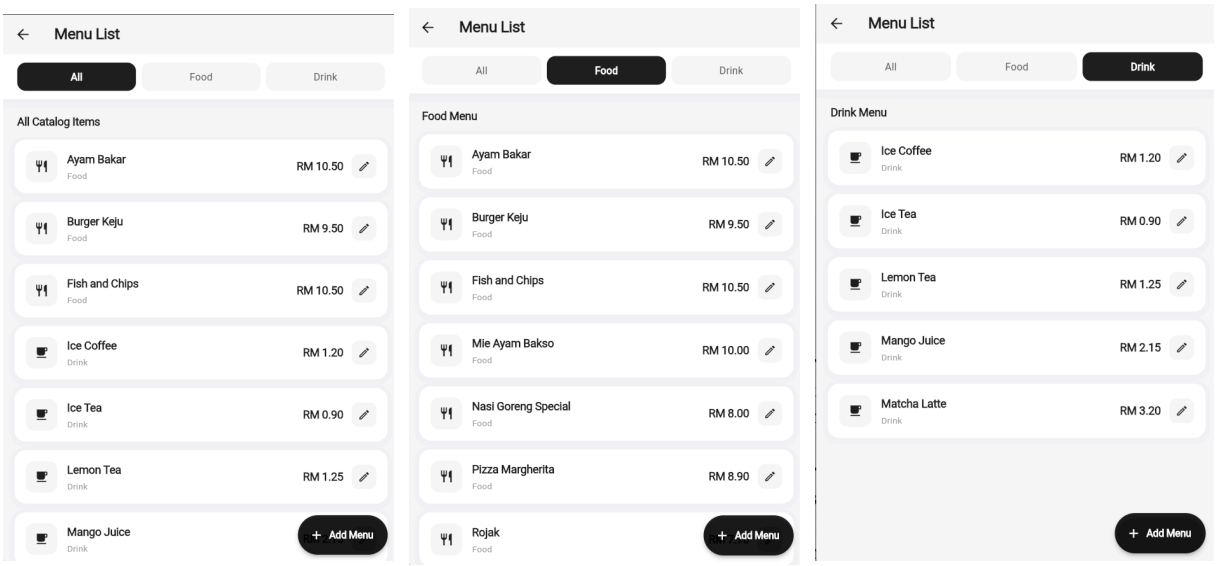

Figure 1: Menu List

The Menu List system functions as the administrative core for catalog management, providing a structured environment where users can perform real-time inventory oversight. From a logic perspective, the interface utilizes conditional tab-based filtering "All," "Food," and "Drink" which dynamically parses the dataset to display only the requested category, thereby reducing user cognitive load during search tasks. Each item is rendered as a distinct card containing the name, price, and category, paired with visual identifiers like fork-and-knife or cup icons to enable immediate recognition of item types. On the backend, the system maintains a persistent connection with the Supabase database, where it performs SELECT queries to fetch the current inventory list. Furthermore, the interface supports CRUD operations by providing an edit pencil icon that links to the

modification logic and a floating action button that triggers an INSERT flow for expanding the menu catalog, ensuring that any administrative change is immediately synchronized across the platform.

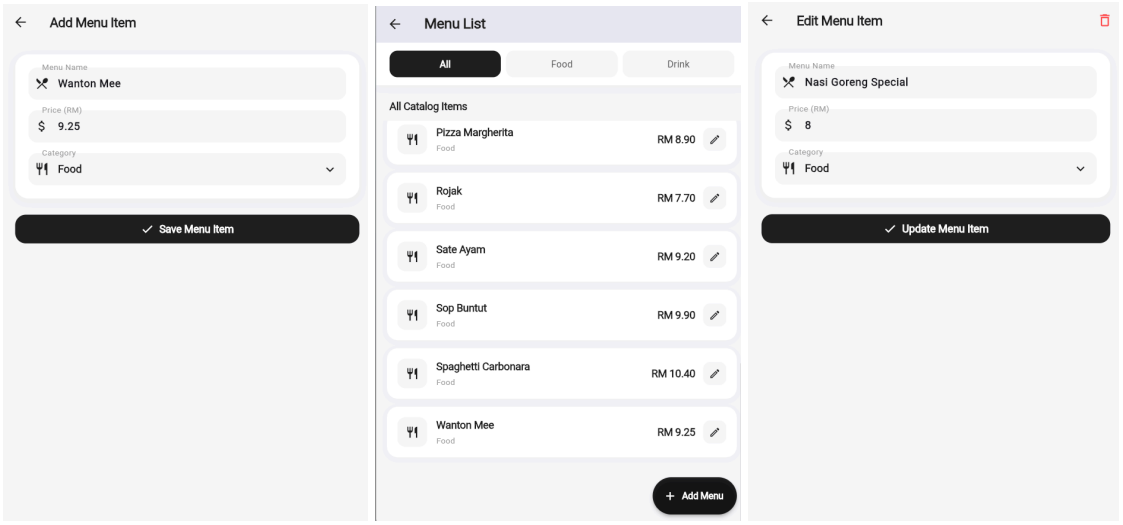

Figure 2: Add/Edit Menu Item

The Add/Edit Menu Item system is designed to provide seamless catalog administration through a unified, state-driven interface. From a system perspective, the interface dynamically adapts based on the user's intent: when triggered via the "+ Add Menu" button, it initializes an empty form for data entry; when triggered via the edit icon on an existing item, it performs a fetch operation to pre-fill the fields with current database values. This dual-purpose design ensures consistency in how menu data is captured and displayed.

The underlying logic relies on a validation-first approach to data handling. Before any record is submitted, the system checks for field completeness, specifically ensuring the menu name, price, and category are properly defined to prevent malformed entries. When the user interacts with the "Save" or "Update" button, the application triggers a state transition that processes these inputs into a structured object, which is then passed to the service layer. This layer handles the translation between the UI state and the database requirements, providing immediate feedback to the user upon a successful write operation.

On the backend, the system utilizes Supabase to maintain a real-time, persistent connection with the application. For new additions, the backend performs an INSERT operation into the menu table, while edit actions utilize an UPDATE operation targeted by the item’s unique identifier (ID). The backend is also configured to handle the deletion of items, signaled by the trash icon in the "Edit Menu Item" view, which executes a DELETE command to remove the record permanently from the database. This architecture ensures that any change made in the Add/Edit interface is immediately reflected across all other views in the application, maintaining synchronization between the administrative catalog and the order-taking screens.

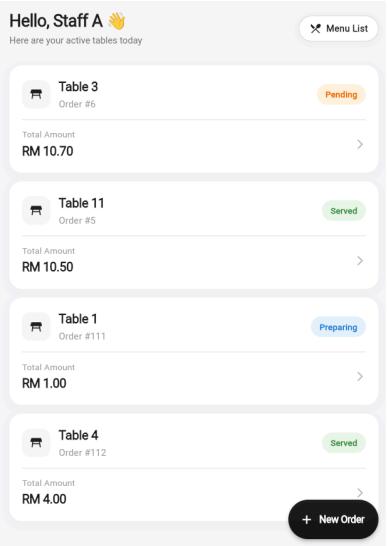

Figure 3: Orders List

The Orders List interface serves as the central dashboard for staff members to monitor real-time dining operations. The system is designed to provide an at-a-glance overview of all active tables, displaying essential details such as the table number, the unique order identification number, and the total accumulated amount for each order. To facilitate rapid operational awareness, the interface incorporates a visual status indicator for each entry, labeled "Pending," "Preparing," "Served", or “Paid” which allows staff to instantly identify the progression stage of every table's request.

From a technical and backend perspective, this screen functions as a high-level view that queries the primary `orders` table in the database. The logic behind this interface employs filtered data retrieval, where the application fetches records based on their lifecycle status to ensure that only relevant, active orders are presented to the staff. Each card in the list acts as a navigation trigger, where the backend supports seamless routing to the specific Order Detail screen, allowing staff to drill down into itemized data when necessary. Additionally, the prominent "+ New Order" floating action button provides a direct, accessible entry point for staff to initiate new transactions, ensuring that the system remains both highly responsive and easy to navigate during busy service hours.

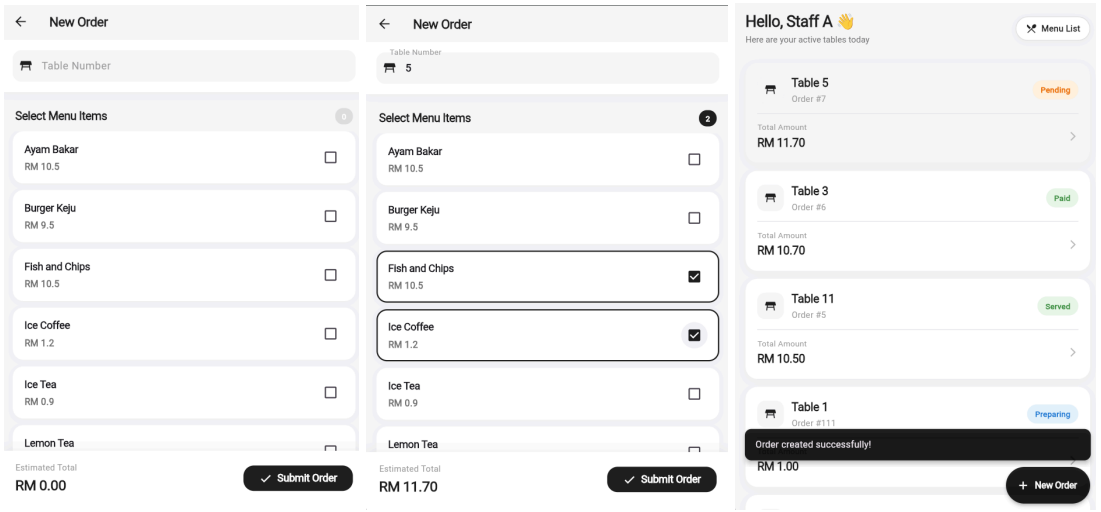

Figure 4: New Order

The New Order interface is designed as an interactive procurement form that streamlines the process of capturing customer requests. From a system perspective, the screen provides a dedicated input field for the table number and a comprehensive list of available menu items, each paired with a checkbox for multi-selection. The logic is built around dynamic state management; as the user selects or deselects items, the system automatically calculates and updates the "Estimated Total" in real-time, providing immediate financial transparency before the order is finalized.

Backend operations for this screen are triggered upon pressing the "Submit Order" button. The application aggregates the selected item IDs, the specified table number, and the calculated total into a structured data object. It then performs two primary backend operations: first, it creates a new record in the orders table to initialize the transaction, and second, it registers individual entries in the order_items table linked to that order ID. Upon successful database write, the backend signals the UI to display a confirmation message "Order created successfully!" and automatically navigates the user back to the primary dashboard, where the new order immediately appears in the active list.

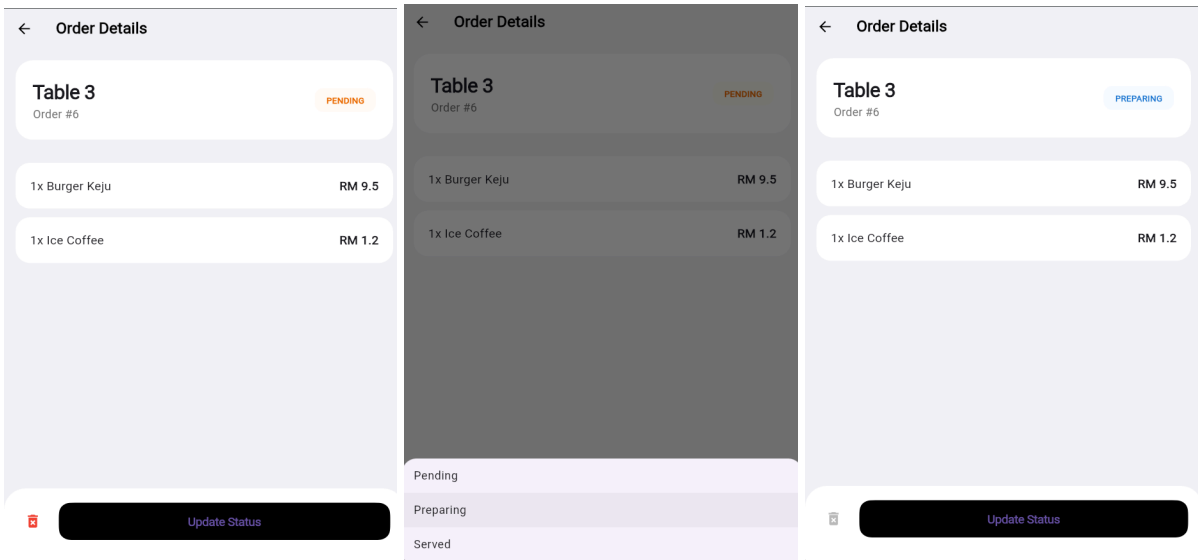

Figure 5.1: Order Detail Before Payment

The Order Details screen functions as the granular management view for specific dining transactions, providing a complete breakdown of ordered items, the final total cost, and a suite of interactive controls for order lifecycle management. From a systems perspective, this screen is engineered to retrieve detailed itemized data linked to a specific Order ID, presenting it in a clear, list-based format for staff verification.

The logic governing this interface is primarily focused on status-based workflow progression. It includes a dynamic "Update Status" button which, when triggered, invokes a modal bottom sheet allowing staff to transition the order through its predefined lifecycle stages, specifically "Pending," "Preparing," and "Served". This transition logic ensures that kitchen and service staff remain synchronized, as updating the status here triggers an immediate update in the central database, reflecting the change across the entire application.

On the backend, this interface maintains robust control over the order state through conditional logic. As demonstrated in the figure, the screen provides a specialized delete action (represented by the trash icon) which is context-aware, meaning it is only active or permitted during early stages of the order process to prevent erroneous cancellations after production has begun. By serving as both a reporting tool for itemized order details and a control panel for operational status updates, this screen ensures accurate, real-time data flow between the front-of-house staff and the database, effectively managing the order from receipt until it reaches the final stage before payment.

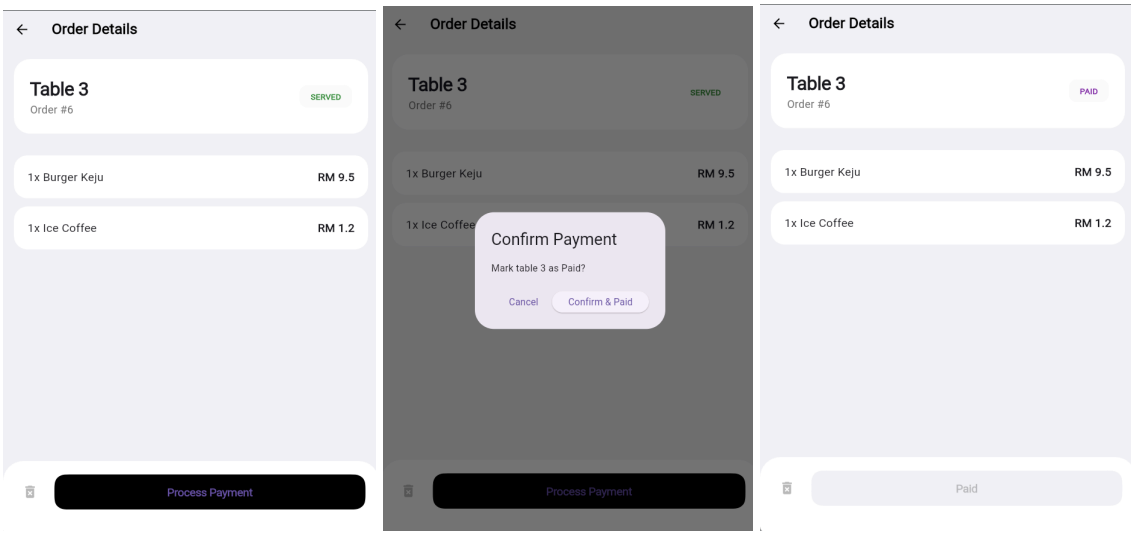

Figure 5.2: Order Detail After Payment

The Order Detail After Payment interface represents the final stage of the transactional lifecycle, designed to securely lock the order state once financial settlement is complete. From a systems perspective, when an order reaches the "Served" status, the primary action button dynamically transitions to "Process Payment," which triggers a confirmation dialog that requires explicit staff validation to proceed. This logic is critical for maintaining financial integrity, as it prevents accidental status changes and ensures that all payments are verified before the system records the transaction as finalized.

On the backend, once the staff confirms the payment, the application executes an UPDATE operation in the Supabase database to change the order status to "Paid". This operation triggers a UI refresh that disables the action button, rendering it inactive with a "Paid" label to indicate that no further modifications to the order items or status are permitted. By transitioning the interface into a locked state, the system ensures that historical transaction data remains immutable, thereby finalizing the order lifecycle and effectively closing out the dining session for that specific table.